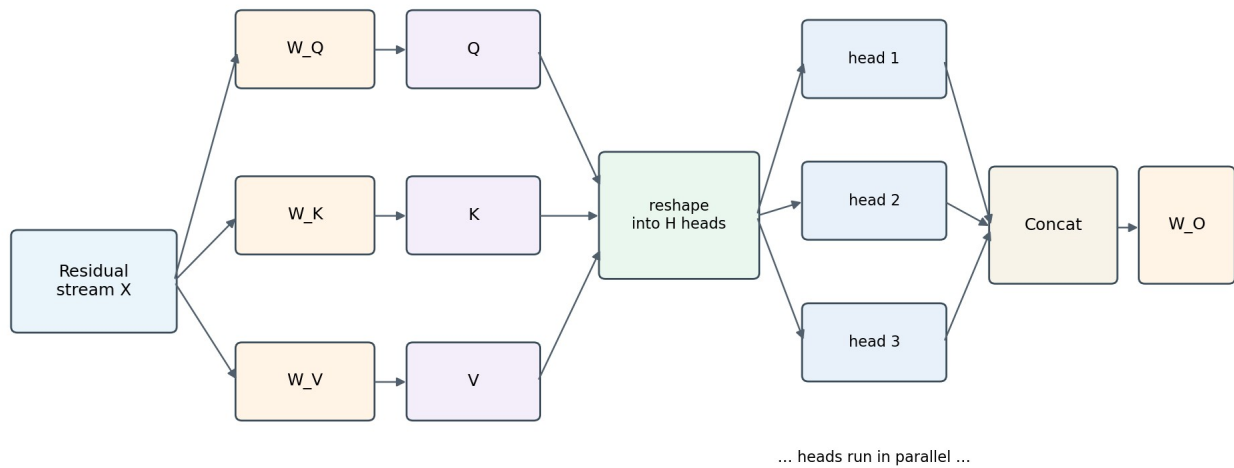


Multi-Head Attention

多头注意力：原理、演化与系统工程

MHA · Head Geometry · Specialization & Redundancy · MQA · GQA · MLA · KV Cache · Tensor Parallel · Gotchas ·
Know-how · Know-why

Multi-Head Attention = parallel learned routing operators + output mixing



Heads are separate before concatenation; W_O is the first standard point where their output channels are linearly mixed.

Research edition · August 2026

核心命题 多头不是“把一个 attention 复制 H 次”。它把固定宽度的表示预算拆成多个并行匹配子空间，再通过 W_O 将不同路由结果重新混合；现代 MQA/GQA/MLA 的演化，本质上是在“head 多样性、KV 状态容量、内存带宽、分布式通信”之间重新分配预算。

目录与阅读路径

1. Executive Summary: 15 个必须记住的结论
2. Multi-Head Attention 的精确定义
3. 为什么需要多个 Head: Know-why
4. Head 数量、Head Dimension 与参数/FLOPs
5. Head 的功能: Specialization、Redundancy 与 Interpretability
6. W_O: 被低估的跨 Head 混合层
7. MHA → MQA: 为什么推理解码先动 K/V
8. MQA → GQA: 质量与带宽的中间点
9. 从 MHA Checkpoint Uptrain 到 GQA
10. KV Cache: 从 Head 配置直接推导显存
11. Prefill vs Decode: 同一个 Attention, 不同硬件瓶颈
12. Tensor Parallel / Context Parallel 的 Head 约束
13. RoPE、QK-Norm 与 Head Geometry
14. FlashAttention / SDPA 与 Multi-Head 布局
15. Head Pruning: 为什么“能剪”不等于“不需要”
16. MLA: 从 KV Head Sharing 到 Latent Compression
17. 2024-2026 Attention Head 新变体
18. 50 个 Gotchas
19. Know-how: 架构设计与部署 Checklist
20. Know-why: 因果原理总表
21. Monitoring / Debugging Playbook
22. Ablation Matrix
23. PyTorch MVP: MHA/GQA/MQA
24. 深度面试与系统设计题
25. 参考文献

阅读建议 从零实现: 2→4→6→23; 做大模型推理: 7→8→10→11→12→16; 做模型研究: 3→5→15→17→22; 做故障排查: 18→21。

1. Executive Summary : 15 个必须记住的结论

#	结论	核心理由
1	MHA 的“多头”是并行子空间，不是多次重复	每个 head 有独立 Q/K/V 投影，可形成不同匹配几何与内容通道。[1]
2	固定 d_model 时，head 越多通常意味着 d_head 越小	经典设置 d_head=d_model/H；这改变单 head 的 rank/容量和 kernel shape。
3	固定 d_model、标准 MHA 下，改变 H 不改变主导参数量	Q/K/V/O 合计约 $4d_{\text{model}}^2$ ；但硬件效率和表达结构会变化。
4	Attention 核心 FLOPs 在固定 d_model 下也近似不随 H 变	$H \cdot L^2 \cdot d_{\text{head}} = L^2 \cdot d_{\text{model}}$ ；更多 heads 主要改变分解方式而非一阶复杂度。
5	Head specialization 与 redundancy 可以同时成立	有些 heads 呈现稳定语言/位置模式；同时大量 heads 可被剪掉而性能变化很小。[2][3][4]
6	Head 的可视化模式不等于因果解释	attention weight 是模型内部信号，不应自动等价于输入重要性。[20]
7	W_O 不是“收尾线性层”	heads 在 softmax/AV 阶段独立，但 concat 后由 W_O 混合回 residual stream；head 的作用要结合 W_O 看。
8	MQA 共享 K/V，而不共享 Q	目标是减少自回归 decode 时反复从 HBM 读取历史 K/V 的带宽。[5]
9	GQA 是 MHA 与 MQA 之间的连续折中	H_q 个 query heads 共享 H_{kv} 个 K/V heads, $1 < H_{kv} < H_q$ 。[6]
10	GQA 最大收益通常在 decode，不是 prefill	Q heads 数量和注意力配对计算仍在；真正大幅减少的是 KV cache 与 decode 读带宽。
11	KV cache 与 H_{kv} 线性缩放	bytes $\approx 2 \cdot B \cdot L \cdot \text{layers} \cdot H_{kv} \cdot d_{\text{head}} \cdot \text{dtype_bytes}$ 。
12	Naive MHA→GQA 平均权重不是免费转换	Ainslie 等提出用均值池化 K/V heads 后再 uptrain；仍需要额外训练恢复质量。[6]
13	TP 世界大小会反过来约束 head 配置	H_q/H_{kv} 是否可整除、KV heads 是否复制，会影响通信和负载均衡。
14	MLA 改变了“缓存什么”	DeepSeek-V2 用低秩 latent KV 联合压缩，进一步减少 KV cache，而不是只减少 KV head 数。[12]
15	2025-2026 对 head 数/共享/低秩的结论仍在快速演化	新论文有价值，但很多只在有限规模验证；生产设计仍以受控 ablation 和目标硬件为准。

2. Multi-Head Attention 的精确定义

对输入 $X \in \mathbb{R}^{(B \times L \times d_{\text{model}})}$, 标准 self-attention 将表示投影到 H 个 head。若 $d_{\text{head}} = d_{\text{model}}/H$:

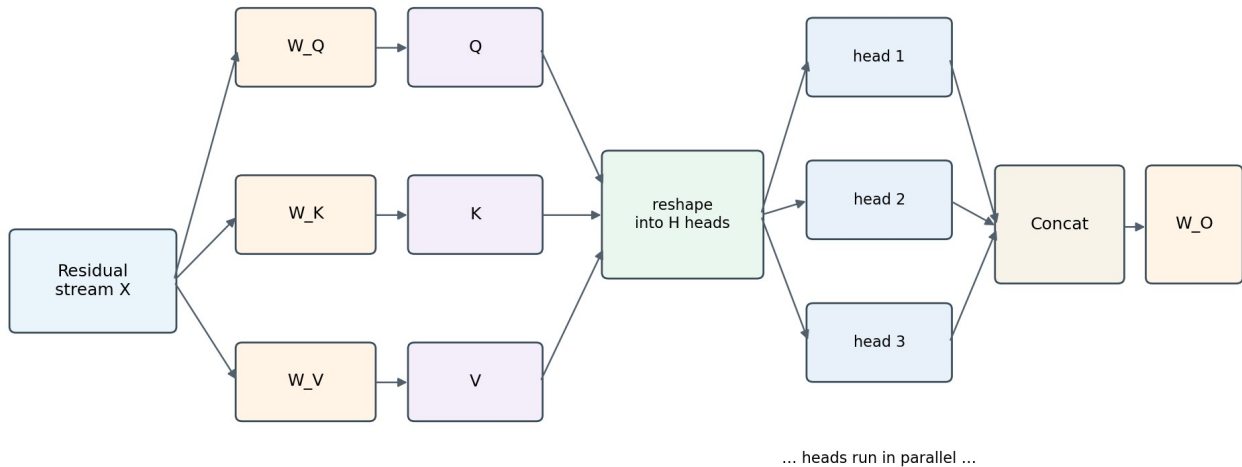
$$Q_h = X W_{Q,h}, \quad K_h = X W_{K,h}, \quad V_h = X W_{V,h}$$

$$A_h = \text{softmax}(Q_h K_h^T / \sqrt{d_{\text{head}}} + \text{mask})$$

$$O_h = A_h V_h$$

$$\text{MHA}(X) = \text{Concat}(O_1, \dots, O_H) W_O$$

Multi-Head Attention = parallel learned routing operators + output mixing



Heads are separate before concatenation; W_O is the first standard point where their output channels are linearly mixed.

2.1 两个“独立”需要区分

- Softmax 前: 每个 head 有自己的 $Q/K/V$ 投影与 attention map, 标准 MHA 不进行显式跨-head 通信。
- 输出后: Concat + W_O 会重新线性混合所有 head 的输出通道, 因此完整模块并非真正独立的 H 个小模型。

Gotcha “Head 3 是语法 head” 这种说法过于强。真正写回 residual stream 的方向由 O_3 与 W_O 对应列共同决定; 只看 attention heatmap 会漏掉 value 与 output projection。

2.2 Head permutation symmetry

如果同时置换 $Q/K/V$ 的 head 顺序, 并对 W_O 的输入通道做相同置换, 函数可以保持不变。这说明 head 编号本身没有天然语义, 跨 checkpoint 比较 “第 7 个 head” 通常没有意义。

3. 为什么需要多个 Head : Know-why

3.1 一个 softmax map 的表达限制

单 head 对每个 query 产生一组归一化权重。多个 head 可以同时维护多组不同的匹配分布：局部、远程、复制、边界、实体、结构等模式可以并行存在，而不必竞争同一个 softmax 概率单纯形。[1]

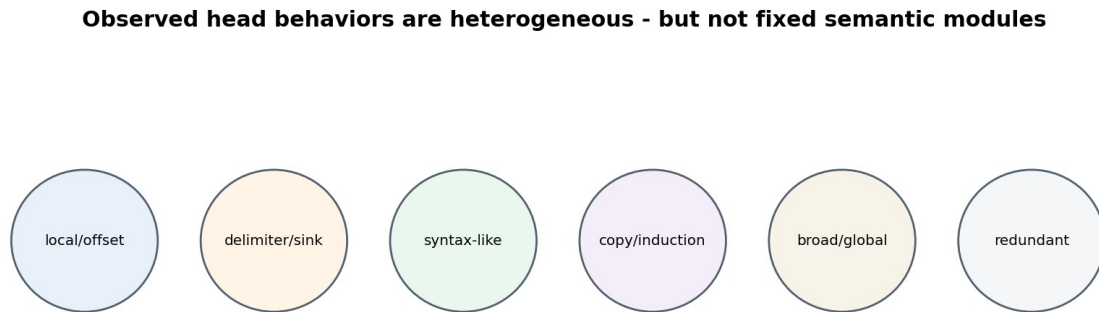
3.2 多个 learned bilinear operators

$$\text{score_h}(i,j) = \mathbf{x}_i^T (\mathbf{W}_{Q,h} \mathbf{W}_{K,h}^T) \mathbf{x}_j / \sqrt{d_{\text{head}}}$$

每个 head 拥有不同的 $\mathbf{M}_h = \mathbf{W}_{Q,h} \mathbf{W}_{K,h}^T$ ，相当于并行学习多种 token-pair 匹配算子。多头的价值更接近“多个路由核”，而不是“多个注意力副本”。

3.3 优化与冗余

2019 年的 pruning 研究发现许多 heads 在训练完成后可移除，但也观察到某些层和某些特化 heads 更关键。[2][3] 这支持一个更细致的理解：训练期的过参数化/多路径可能有助于优化，最终函数却可能只依赖其中一个子集。



Pruning studies show redundancy; analysis studies find specialized patterns. Both can be true at the same time.

Know-why “冗余”可能是训练过程的保险，而不是纯浪费：多个 heads 能提供备选路由、对称破缺路径和梯度通道。post-hoc 可剪性不能直接等价于最优预训练架构。

4. Head 数量、Head Dimension 与参数/FLOPs

4.1 标准参数量

若 Q/K/V 的总输出宽度都等于 d_{model} ，且 W_O 为 $d_{\text{model}} \times d_{\text{model}}$ ：

$$\text{Params_MHA} \approx 3 d_{\text{model}}^2 + d_{\text{model}}^2 = 4 d_{\text{model}}^2 \quad (\text{bias omitted})$$

所以在经典约束 $H \cdot d_{\text{head}} = d_{\text{model}}$ 下， H 本身不改变一阶参数量。

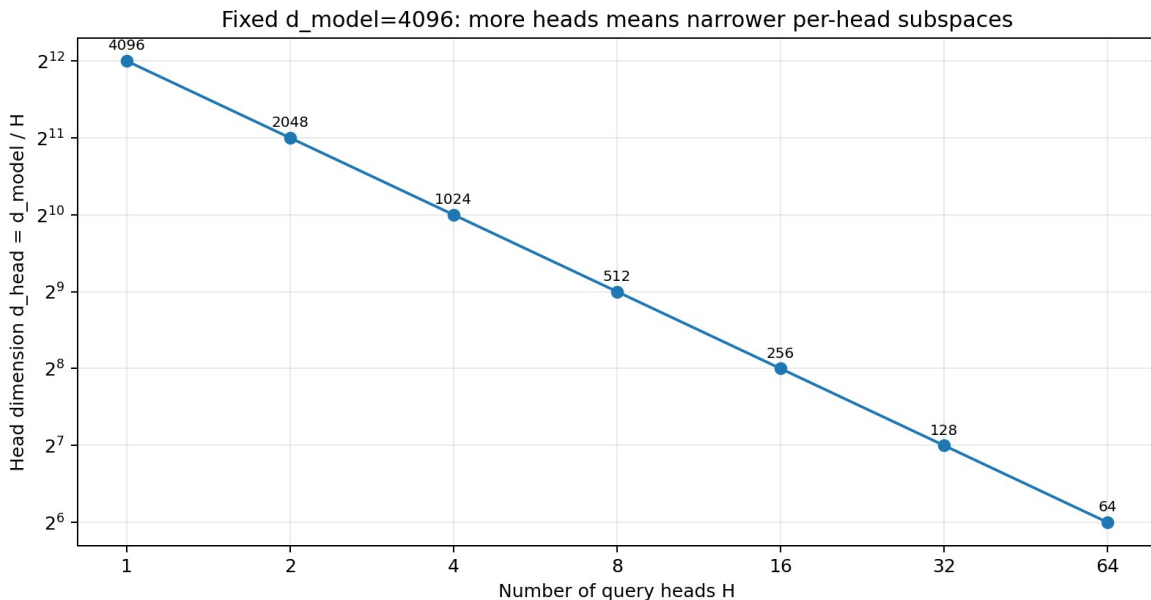
4.2 Attention 主体 FLOPs

$$\text{QK}^T: H \cdot L^2 \cdot d_{\text{head}} = L^2 \cdot d_{\text{model}}$$

$$\text{AV}: H \cdot L^2 \cdot d_{\text{head}} = L^2 \cdot d_{\text{model}}$$

固定 d_{model} 时，增加 head 数不会降低二次 attention 的一阶 FLOPs，但会改变 GEMM/tile 形状、并行度和 kernel overhead。

4.3 Head dimension 是隐藏的架构超参数



标准 Transformer 常把 d_{head} 绑定到 d_{model}/H ，但现代模型/实现中 head_dim 可以显式设定。过小 d_{head} 会限制单 head 的匹配秩与表示宽度；Bhojanapalli 等从表达能力角度分析了 head size 造成的低秩瓶颈。[7]

Gotcha “更多 heads 一定更强”没有普适结论。固定宽度下， $H \uparrow$ 会让 $d_{\text{head}} \downarrow$ ；功能多样性与单-head 容量之间存在 trade-off，还受数据、深度、RoPE、kernel 支持影响。

5. Head 的功能：Specialization、Redundancy 与 Interpretability

5.1 观察到的 specialization

BERT/MT 分析工作观察到部分 heads 对固定位置偏移、分隔符、句法依赖、共指等模式有较强对应关系。[3][4] 这些现象说明 head 可以形成相对稳定的功能偏置。

5.2 同时存在大量 redundancy

Michel 等发现，训练完成的模型中大量 heads 可以在推理时移除而不显著损害性能；Voita 等也发现专门化 heads 往往最后才被剪掉。[2][3]

5.3 为什么两者不矛盾

- “存在关键 head” 不等于 “所有 head 都关键”。
- “某个 head 可被剪” 不等于它训练期间没贡献；其他 heads/MLP 可能已经形成补偿。
- Head 功能是分布式的：attention map、V 内容、W_O 写入方向和后续层共同决定效果。

5.4 Attention heatmap 不是完整解释

Jain & Wallace 发现 attention 权重与其他重要性度量可能不一致，且不同 attention 分布可产生相似预测。[20] 对 LLM head 做因果解释时，更可靠的方法是 activation patching、head ablation、path patching 等干预，而不是只看热图。

实践建议 把 attention visualization 当“候选机制发现工具”，再用干预实验验证因果性。

6. W_O : 被低估的跨 Head 混合层

各 head 输出 $O_h \in \mathbb{R}^{(L \times d_{\text{head}})}$ 拼接成 $O_{\text{cat}} \in \mathbb{R}^{(L \times H d_{\text{head}})}$, 然后:

$$Y = O_{\text{cat}} W_O$$

W_O 决定每个 head 的哪些 value-space 方向被写回 residual stream, 以及不同 head 的输出如何线性组合。

6.1 为什么不能只研究 attention matrix

$$Y_i = \sum_h \sum_j \alpha_h(i,j) \cdot (x_j W_{V,h}) \cdot W_{O,h}$$

从最终输出看, α 只是乘法链的一部分。一个 attention 权重很大的 token, 如果对应 V/W_O 方向幅度小, 对 residual stream 的实际影响仍可能有限。

6.2 Head pruning 的真正速度收益

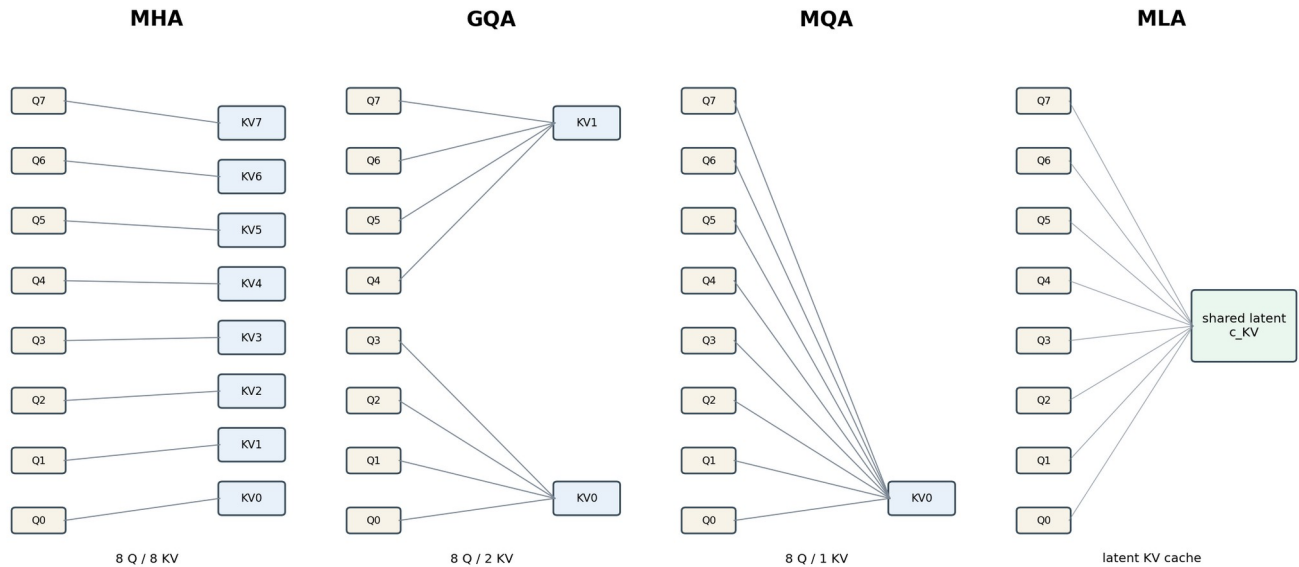
仅把某个 head 的 mask 设为 0, 并不会自动让 fused dense GEMM 变小。要获得真实延迟收益, 需要结构化删除 Q/K/V 对应通道和 W_O 对应输入列, 并使用能够利用新 shape 的 kernel。

Gotcha “模型精度不变”与“系统速度变快”是两次不同实验。结构化 sparsity 若没有对应 kernel, 可能只减少数学工作量, 不减少 wall-clock latency。

7. MHA → MQA : 为什么推理解码先动 K/V

Shazeer 2019 提出 Multi-Query Attention: 保留多个 Q heads, 但所有 query heads 共享一组 K/V。[5]

$$H_q = H, H_{kv} = 1$$



7.1 Decode 的核心瓶颈

自回归 decode 每次只生成一个新 token。当前 token 的 Q/K/V 投影很小，但每层必须读取所有历史 token 的 K/V。长上下文和大 batch 下，这经常是 memory-bandwidth-bound。

7.2 MQA 为什么有效

- 把历史 KV 状态从 H 份降到 1 份。
- 减小 KV cache, 可提高可用 batch size。
- 减少每个 decode step 从 HBM 读取的字节数。

7.3 代价

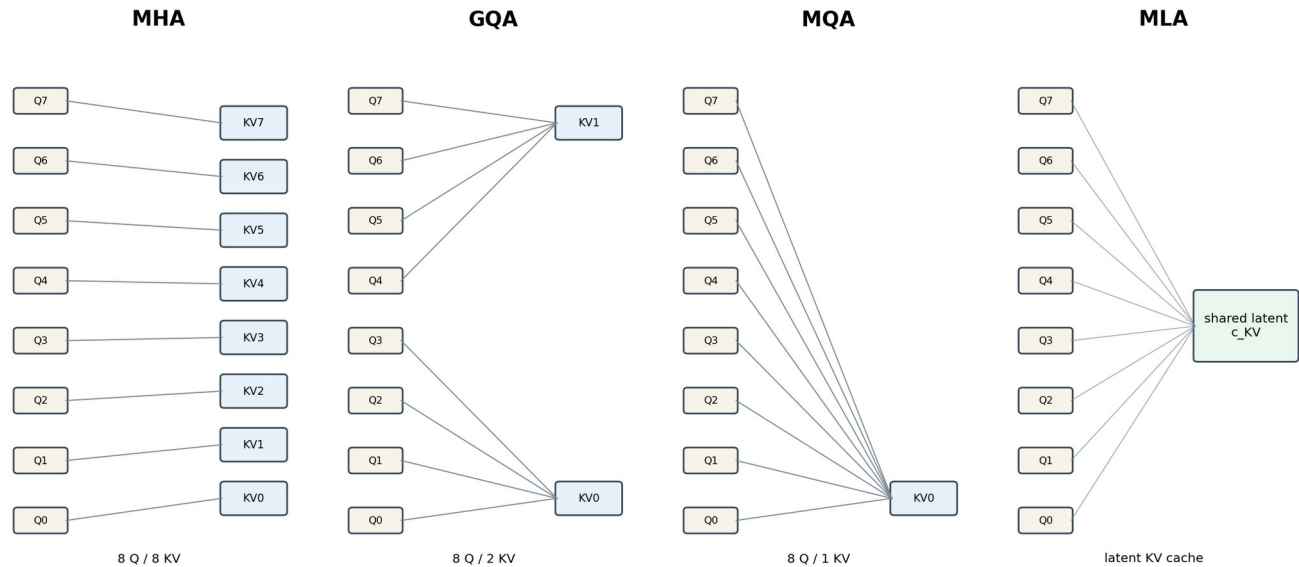
所有 query heads 共享同一 K/V 索引与内容表示，降低 K/V 多样性，因此可能带来质量损失。[5][6]

Know-why MQA 的设计目标主要是“少搬数据”，不是“少算 QK 的所有 query-token 关系”。因此它对 decode 的提升通常远大于对训练/prefill 的提升。

8. MQA → GQA : 质量与带宽的中间点

Grouped-Query Attention 使用 H_q 个 query heads 和 H_{kv} 个 KV heads, 其中 $1 < H_{kv} < H_q$ 。每组 query heads 共享一个 KV head。[6]

group_size = H_q / H_{kv} (common implementation requires integer)



8.1 连续谱

形式	H_q	H_{kv}	KV 多样性	KV cache	典型定位
MHA	H	H	最高	1.0x	训练/质量基线
GQA	H	$1 < G < H$	中	G/H	现代 LLM 常用折中
MQA	H	1	最低	1/H	极致 decode 带宽

8.2 为什么 GQA 成为主流折中

Ainslie 等报告 GQA 在 uptraining 设置下能够接近 MHA 质量, 同时速度接近 MQA。[6] 后续 Llama 2 大模型、Mistral 7B、Qwen2 等都采用 GQA 以提高推理可扩展性。[9][10][11]

8.3 参数量也会下降, 但不是主要价值

$Q: d \cdot (H_q d_h); \quad K, V: 2 d \cdot (H_{kv} d_h); \quad O: (H_q d_h) \cdot d$

当 $H_q d_h = d$ 时, GQA 会缩小 K/V projection 参数; 但对大模型而言 FFN 通常占更多参数, 真正系统级收益往往来自 cache/bandwidth。

9. 从 MHA Checkpoint Uptrain 到 GQA

Ainslie 等提出从已有 MHA checkpoint 转成 MQA/GQA，再用约原始预训练计算的一小部分进行 uptraining。[6]

9.1 常见初始化：KV head pooling

若把若干原始 K/V heads 合并成一个 group，可对组内 K/V projection 权重做均值池化作为初始化：

$$\mathbf{W}_{\mathbf{K},\text{new},g} = \text{mean}_{\{h \text{ in group } g\}}(\mathbf{W}_{\mathbf{K},h}), \quad \mathbf{W}_{\mathbf{V},\text{new},g} = \text{mean}_{\{h \text{ in group } g\}}(\mathbf{W}_{\mathbf{V},h})$$

9.2 为什么不能只平均然后直接部署

- 不同 heads 的功能未必对齐，平均会产生表示干扰。
- Q heads 已经适配原始独立 K/V；共享后匹配分布改变。
- RoPE/QK-Norm/scale 与 cache layout 需要保持一致。
- 下游 fine-tune 不能替代足够广覆盖的 continued pretraining。

Gotcha MHA→GQA 是模型架构转换，不只是 checkpoint reshape。若没有重新训练，困惑度、长上下文和特定能力都可能出现隐性回退。

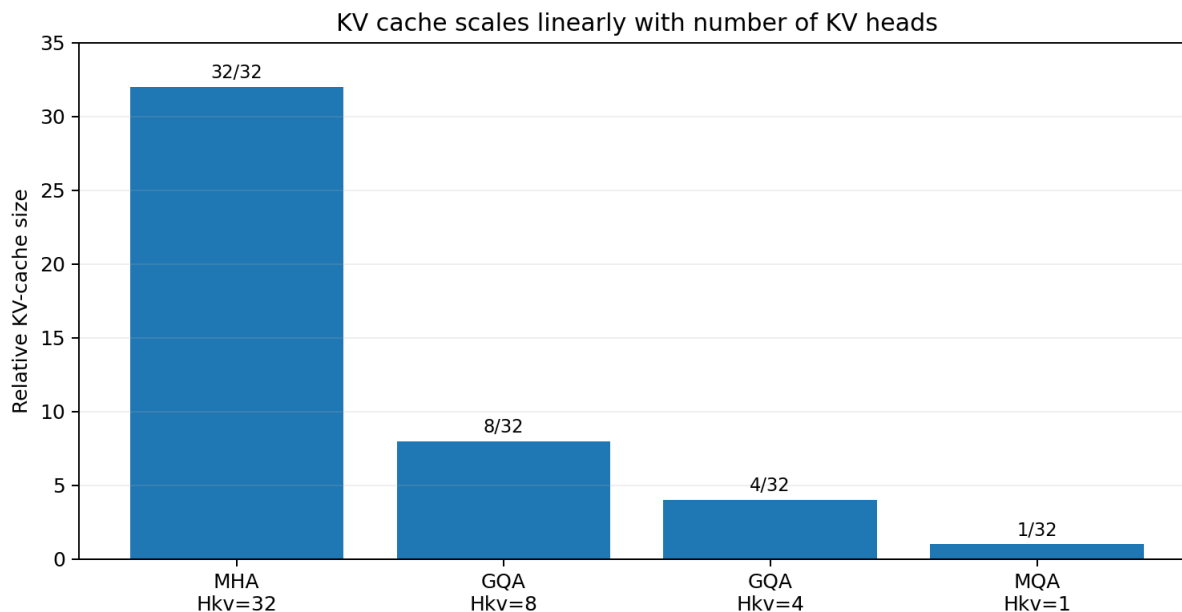
9.3 Ablation 必须看什么

至少同时比较 validation loss、长上下文任务、decode tokens/s、最大 batch、实际 KV bytes、不同序列长度的 p50/p95 latency；不要只看一个下游 benchmark。

10. KV Cache : 从 Head 配置直接推导显存

$$\text{KV_bytes} = 2 \cdot B \cdot L \cdot N_{\text{layers}} \cdot H_{\text{kv}} \cdot d_{\text{head}} \cdot \text{bytes_per_element}$$

前面的 2 对应 K 和 V。对于相同 d_{head} / 层数 / 上下文, KV cache 与 H_{kv} 线性缩放。



10.1 示例

假设 32 层、 $H_q=32$ 、 $d_{\text{head}}=128$ 、BF16(2 bytes)、context=128K、batch=1:

$$\text{MHA}(H_{\text{kv}}=32): 2 \cdot 32 \cdot 131072 \cdot 32 \cdot 128 \cdot 2 \approx 64 \text{ GiB}$$

$$\text{GQA}(H_{\text{kv}}=8): \approx 16 \text{ GiB}$$

$$\text{MQA}(H_{\text{kv}}=1): \approx 2 \text{ GiB}$$

这是纯 KV 张量的数量级示例, 实际引擎还会有 block metadata、对齐、碎片、其他激活/权重等开销。

Gotcha KV cache 预算应该按“并发请求 $\times$ 实际上下文分布”计算, 而不是只拿单请求最大 context。Serving 中可支持的并发往由 KV cache 池决定。

11. Prefill vs Decode : 同一个 Attention，不同硬件瓶颈

11.1 Prefill

- 一次处理 L 个 prompt tokens，矩阵乘较大，GPU 更容易达到高算力利用率。
- Attention 核心仍要处理所有 query heads；GQA 对核心 QK/AV 二次项的减少有限。
- K/V projection 与写 cache 会更小，但不是全部成本。

11.2 Decode

- 每步 query length ≈ 1 ，GEMM 变“小而窄”，算力利用率降低。
- 必须读取很长的历史 KV；因此 bytes/token 成为关键。
- GQA/MQA/MLA 的价值在此放大。

11.3 Arithmetic intensity 直觉

Decode 时每加载一个历史 K/V 元素只服务很少的新 query，数据复用差；减少 H_{kv} 可以直接提升有效 arithmetic intensity。

Know-how 评估 attention variant 时必须把 TTFT (prefill) 和 TPOT/ITL (decode) 分开测；平均 tokens/s 会掩盖完全不同的瓶颈。

12. Tensor Parallel / Context Parallel 的 Head 约束

12.1 Query heads 的自然切分

在 column-parallel QKV projection 中, Q heads 常沿 head/channel 维分给不同 TP ranks。理想情况是:

$$H_q \% TP_world_size == 0$$

12.2 GQA 的 KV heads 可能不够分

当 $H_{kv} < TP \text{ world size}$ 时, 不能简单一 rank 一个独立 KV head。常见实现会复制 KV heads 到多个 rank, 或使用更复杂的 sharding。复制不会改变数学结果, 但会削弱理论 cache/bandwidth 节省。

12.3 W_O 与通信

每个 rank 计算部分 head 输出后, output projection / row parallel 通常需要 reduce-scatter 或 all-reduce。Head 配置因此影响的不只是本地 kernel, 也影响跨 GPU 通信。

12.4 Context Parallel

长上下文下, 可沿 sequence 维切分 K/V。此时 GQA 较小的 K/V feature size 会降低部分通信量, 但需要正确处理 causal 边界、RoPE position id 和 online softmax 的全局归约。

Gotcha “单卡 KV cache 降了 4x” 不等于 “8 卡集群流量也降 4x”。TP 复制、CP all-to-all、page layout 都会改变实际收益。

13. RoPE、QK-Norm 与 Head Geometry

13.1 RoPE 发生在每个 Q/K head 的坐标对上

RoPE 通常要求 rotary dimension 与 head dimension 的布局一致。改变 H 或 d_head 会改变每个 head 可用于位置旋转的维度数，不能只复制旧配置。

13.2 Partial RoPE

一些架构只旋转 head 中一部分维度。此时“head_dim 改了但 partial_rotary_factor 不变”会改变绝对 rotary dims，从而改变位置容量。

13.3 QK-Norm

QK-Norm 对每个 head 的 Q/K 向量做归一化或尺度控制，目标是稳定 attention logits。它与 head_dim、softmax temperature、RoPE 频率共同决定匹配几何。

13.4 GQA 不意味着共享 Q

GQA 中 query heads 仍然独立；共享的是 K/V。多个 Q heads 可以用不同查询几何读取同一 KV 表示，这是 GQA 维持质量的重要来源。

Gotcha 调 head 数时如果同时改变 d_head、RoPE theta、rotary dims、QK-Norm scale，就已经不是“只做 head-count ablation”。必须一次只改一个自由度。

14. FlashAttention / SDPA 与 Multi-Head 布局

FlashAttention 改变执行顺序，不改变 MHA 数学定义；但 kernel 对 head_dim、dtype、causal、GQA、dropout 等组合有具体支持边界。

14.1 为什么 head_dim 会影响 kernel

- Tile shape 与 SRAM/register 占用由 d_head 决定。
- 过小 head_dim 可能让 GEMM 太窄，kernel launch/调度开销占比上升。
- 过大 head_dim 会提高每 tile 状态与寄存器压力。

14.2 GQA 的逻辑 broadcast vs 物理复制

数学上，一个 KV head 可以服务多个 Q heads。高效 kernel 应在 tile 内复用 K/V，而不是先把 K/V 物理 repeat 成 H_q 份；后者会把 cache 优势重新吃掉。

14.3 Backend fallback

某些 mask、head_dim、dtype、dropout 或非连续 layout 组合可能从 fused kernel fallback 到较慢实现。性能回归要先确认实际 backend，而不是只看 Python 层代码。

Know-how benchmark 时记录 kernel/backend 名、head_dim、H_q/H_kv、sequence length、dtype、layout(contiguous/strided)，否则结果不可复现。

15. Head Pruning : 为什么“能剪”不等于“不需要”

15.1 经典结果

Michel 等与 Voita 等都展示了大比例 attention heads 可在训练后剪除而只带来小幅性能下降。[2][3]

15.2 三个常见错误推论

错误推论	为什么不成立
很多 head 可剪 → 训练时只需 1 head	训练期优化路径、冗余和对称破缺可能依赖多头。
剪掉不掉分 → head 完全没用	任务评测可能不覆盖其功能；其他组件也可能补偿。
mask 掉 head → 推理一定变快	dense fused kernel 不会因为逻辑 mask 自动缩小。

15.3 更严谨的 pruning 实验

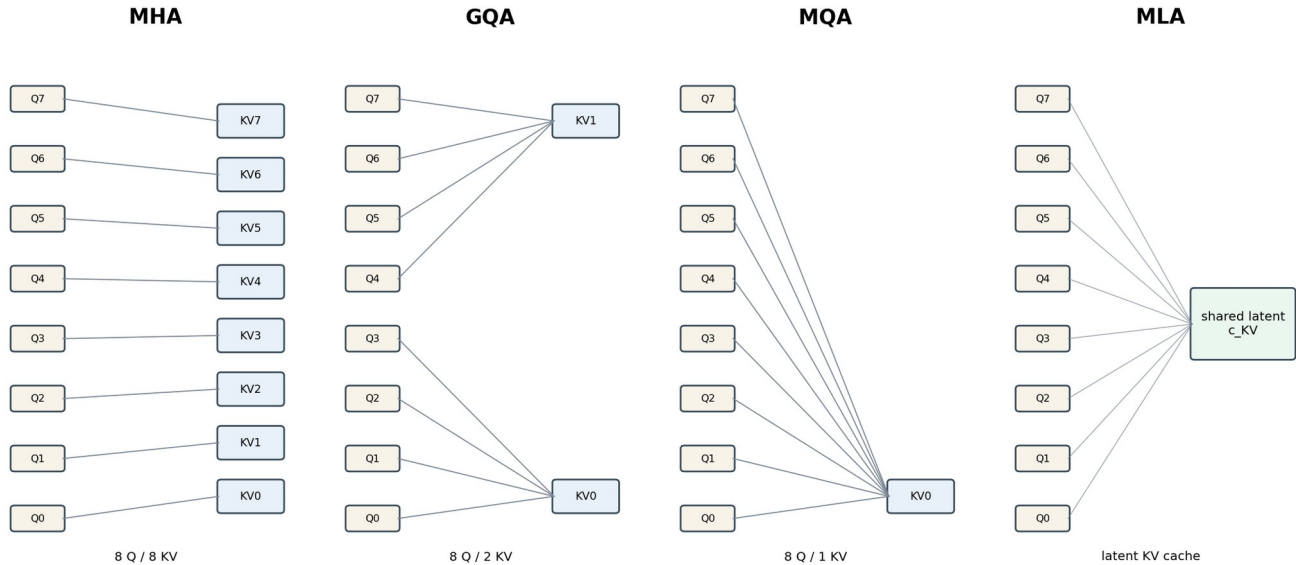
- 跨多个数据集/任务测量，而非只看平均 benchmark。
- 结构化删除权重并重新编译/重建 kernel。
- 测 p50/p95 latency、吞吐和显存，而非只算 FLOPs。
- 区分 zero-shot pruning、短期 recovery fine-tune 与长期 retraining。

16. MLA : 从 KV Head Sharing 到 Latent Compression

DeepSeek-V2 引入 Multi-head Latent Attention (MLA), 核心不再是“多少个 KV heads”, 而是先把每个 token 的 K/V 信息压缩到低维 latent c_{KV} , 再按需要恢复与 query 交互。[12]

$$c_{KV} = x W_{DKV} \text{ (low-rank compressed latent)}$$

K/V-related representations = projections from c_{KV} (+ decoupled positional branch)



16.1 与 GQA/MQA 的本质差异

机制	缓存单位	压缩方式	主要风险
MHA	每 token $\times$ 多 KV heads	无共享	cache 最大
GQA	每 token $\times$ 少量 KV heads	head sharing	KV 多样性下降
MQA	每 token $\times$ 1 KV head	极致 head sharing	质量风险更高
MLA	每 token $\times$ latent state	low-rank joint compression	实现/并行更复杂

16.2 为什么 RoPE 要解耦

如果把完整位置相关 K 直接缓存, 低秩 latent 的吸收/重参数化会受旋转位置项限制。DeepSeek-V2 采用解耦的 RoPE 分支以兼顾 latent cache 与位置表示。[12]

Gotcha MLA 的理论 cache 压缩不能直接等价于所有硬件上的线性吞吐提升; TP sharding、latent head 复制、kernel 支持和矩阵吸收策略都会影响实际速度。

17. 2024-2026 Attention Head 新变体（新兴研究）

下面列出的方向值得关注，但与 MHA/GQA/MQA/MLA 的成熟度不同。报告将其作为研究前沿，而非生产默认方案。

方向	核心思想	潜在收益	主要不确定性
DCMHA (2024)	动态跨-head 组合 attention scores/weights	增强 head 交互与表达能力	额外算子与大规模验证范围 [13]
DHA (2024)	学习 decoupled-head 共享结构	在 KV cache 与 head diversity 间自适应折中	训练/转换复杂度 [14]
MoH (2024)	把 attention heads 视作 mixture-of-heads	条件激活、减少无效 head 计算	路由稳定性与 kernel [15]
Cost-optimal GQA (2025)	联合 context/硬件成本搜索 H_kv	更系统地选 GQA 组数	主要在有限模型规模验证 [16]
TransMLA (2025)	将 GQA/MHA checkpoint 转成 MLA	复用已有模型、减 cache	转换与恢复训练成本 [17]
Head-count theory (2025/26)	研究“足够 heads”与表达效率	解释多头为何有必要	理论任务到真实 LLM 的外推 [18]
LRKV / MLRA (2026)	低秩或多 latent-head KV 结构	兼顾 cache、质量、TP 可分片	非常新，需更广泛复现 [19]

研究原则 不要因为论文报告“2x/3x”就直接替换生产 attention。必须要在你的模型宽度、上下文、TP 拓扑、batch 分布和 kernel 上重新测。

18.50 ↑ Gotchas

#	Gotcha	后果 / 正确理解
1	<code>d_model % H_q != 0</code>	经典 reshape 直接失败；除非架构显式支持独立 head_dim。
2	误把 <code>d_head</code> 当 <code>d_model</code>	scale、RoPE、cache shape 都会错。
3	<code>H_q % H_kv != 0</code>	多数 GQA kernel 要求整数 group size。
4	把 GQA 的 fused QKV 三等分	K/V 宽度比 Q 小，split size 必须按真实 shape。
5	逻辑 repeat KV 变成物理 repeat	把 cache/bandwidth 优势抵消。
6	改变 H 但忘了改 checkpoint layout	head 通道排列可能交错而非 Q...K...V。
7	W_O 输入列没随 head pruning 删除	数值/shape 错或没有真实加速。
8	把 head index 当固定语义	head permutation symmetry；跨 seed/模型不可直接对号。
9	只看 attention heatmap 判因果	忽略 V、W_O 与后续层。
10	以 post-hoc pruning 推导预训练 head 数	训练动态与最终冗余不是一回事。
11	以为 H 增加会降低 $O(L^2d)$ FLOPs	固定 <code>d_model</code> 下 leading term 基本不变。
12	以为 GQA 会让 prefill 同比例加速	核心 query-attention 计算仍在，收益主要是 KV/bandwidth。
13	KV cache 公式漏掉 K/V 的 factor 2	显存预算低估 2x。
14	KV cache 预算漏 batch	Serving 并发一上来立刻 OOM。
15	只按 max context 算单请求	实际容量取决于请求长度分布和 allocator。
16	TP ranks > <code>H_kv</code> 时忽略 KV 复制	真实内存/流量高于理论。
17	<code>H_q</code> 不能整除 TP world size	需要不均匀 shard 或复制，kernel 复杂。
18	beam search/reorder 只重排 Q 不重排 cache	生成结果静默错误。
19	paged KV metadata 与 <code>H_kv</code> 不一致	地址计算错，可能仅在长序列触发。
20	chunked prefill position/cache offset 错	短测试正常，跨 chunk 结果漂移。
21	head_dim 变化但 RoPE rotary_dim 不变	位置几何被意外改变。
22	partial RoPE 用比例还是绝对维度混淆	迁移 checkpoint 容易错。
23	QK-Norm 参数粒度错	per-head/per-dim 的实现差异会改变结果。
24	scale 使用 <code>sqrt(d_model)</code>	softmax 温度严重偏差。
25	Q/K/V bias 配置与 checkpoint 不匹配	加载看似成功但数值不一致。

#	Gotcha	后果 / 正确理解
26	MHA→GQA 只平均 K/V 不 uptrain	质量可能明显下降。
27	pooling group 顺序错	本应相邻/特定 heads 的映射被破坏。
28	量化所有 heads 共用一个 scale	少量 outlier heads 可主导误差。
29	KV cache 量化只测短 context	误差可能随读取次数/距离累积。
30	FlashAttention backend fallback 未发现	吞吐突然下降但模型代码没变。
31	head_dim 不受目标 kernel 高效支持	理论架构好，实际 kernel 很慢。
32	非 contiguous head layout 触发额外 copy	隐藏的 memory bandwidth 成本。
33	训练时 head dropout，推理忘记关闭/缩放	分布不一致。
34	mask shape 广播到错的 head 维	某些 heads 看到了不该看的 token。
35	cross-attention 错把 <code>H_q=H_kv</code> 当必然	encoder/decoder 可使用不同 head 配置。
36	只比较参数量忽略 KV cache	LLM serving 的主瓶颈可能完全在状态而非权重。
37	只比较 cache 忽略 output projection	MHA/GQA 的权重与 compute trade-off 仍要完整统计。

38	把 MQA 说成 “单头 attention”	它仍有多个 query heads，只是共享 K/V。
39	把 GQA 说成 “减少 query heads”	GQA 核心是减少 KV heads。
40	把 MLA 等同于 MQA	MLA 是 latent compression，不是单 KV head。
41	MLA latent cache 在 TP 上不可自然切分	可能导致复制和带宽瓶颈。
42	转换 attention 架构后不重做 calibration	量化/FP8 scale、cache scale 都可能失效。
43	只测 tokens/s 不分 TTFT/TPOT	掩盖 prefill/decode trade-off。
44	只测 batch=1	GQA 的可用 batch 容量收益被低估。
45	只测平均长度	长尾请求才可能决定显存和 p95 延迟。
46	head pruning 只在单 benchmark 验证	少数能力/语言可能被定向损伤。
47	认为更多 heads 会自动 specialization	很多 heads 可能学到相似或 sink 模式。
48	认为每个 head 的输出幅度相同	V/W_O norm 可差很多，attention map 相同也不代表贡献相同。
49	模型配置写 H_kv，但 serving engine 未正确读取	退化成错误 repeat/reshape 或直接不兼容。
50	把论文的理论 speedup 当实际 latency	HBM、kernel、通信、batch、allocator 都会改变实现收益。

19. Know-how : 架构设计与部署 Checklist

19.1 从零设计模型

- 先确定 d_{model} 与目标 hardware-friendly d_{head} , 再得到 H_q , 而不是先拍脑袋定 H 。
- 把 H_{kv} 视为 serving 超参数: 结合 max context、并发、TP world size 与质量预算搜索。
- 长上下文模型优先在真实 context distribution 上测 KV bytes 和 decode bandwidth。
- RoPE/QK-Norm/partial rotary 与 head_dim 联合记录在 config, 不要靠隐含默认值。
- 对 MHA/GQA/MQA 至少做等 token budget 的小模型 proxy ablation。

19.2 从已有 MHA 转 GQA

- 固定 Q heads 与 W_O , 按明确 group mapping pool K/V。
- 保存转换 manifest: old_head $\rightarrow$ new_kv_group 的映射。
- continued-pretrain 恢复, 而不是只做任务 SFT。
- 重新做长上下文/多语言/代码/工具调用评测。
- 重新做 quantization calibration 与 serving parity。

19.3 Serving

- 理论 KV bytes 与 engine 实测 allocated bytes 对账。
- 分别记录 prefill tokens/s、decode tokens/s、TTFT、TPOT、p95。
- 在 batch=1 与高并发两端都测。
- 检查实际 kernel/backend 与是否发生 KV physical repeat。
- 对 TP/CP 拓扑单独测通信占比。

经验法则 Head 架构不是单纯“模型质量超参数”, 而是模型-系统 co-design 参数。先写出目标负载: context、batch、latency、GPU 拓扑, 再决定 H_q/H_{kv} 。

20. Know-why : 因果原理总表

问题	Know-why
为什么多头而不是单头?	多个独立 softmax 路由分布 + 多个 learned bilinear matching spaces, 可并行表示不同关系。
为什么固定 d_model 时参数不随 H 变?	总 Q/K/V 输出宽度仍是 d_model, 只是通道重新分组。
为什么更多 heads 不是免费容量?	每 head 变窄, 单 head rank/内容通道容量下降, 且 kernel 变碎。
为什么 MQA 共享 K/V 而不共享 Q?	decode cache 存的是历史 K/V; Q 只属于当前 query, 不需要长期缓存。
为什么 GQA 比 MQA 通常质量更好?	保留多个 KV 子空间, 减少所有 Q heads 竞争同一 K/V 表示的限制。
为什么 GQA 对 decode 更重要?	decode 的关键成本是反复读取历史 cache; prefill 更接近大矩阵算力问题。
为什么 head pruning 不保证训练时少 head 更好?	过参数化可以改善优化与路径探索, 训练后再冗余化。
为什么 heatmap 不能直接解释模型?	最终贡献 = attention weight $\times$ V 表示 $\times$ W_O 写入 $\times$ 后续非线性。
为什么 W_O 很重要?	它把 head-local 内容重新编码为 residual-stream 方向, 决定 head 输出如何被下游消费。
为什么 H_kv 与 TP world size 要一起设计?	head 是天然 shard 单位; 不足时必须复制, 破坏内存/带宽理想值。
为什么 MLA 比 GQA 更激进?	它不只是共享 heads, 而是压缩每 token 的长期 KV state 维度。
为什么新 attention variant 必须在目标硬件测?	很多收益来自 memory hierarchy 与 kernel, 而不是抽象 FLOPs。

21. Monitoring / Debugging Playbook

21.1 训练期每层/每 head 监控

指标	异常信号	可能原因
Q/K RMS	少数 heads 持续爆大	logit saturation、norm drift、QK-Norm/scale 配置
Attention entropy	过早接近 0	softmax 极尖、scale/position 问题
Attention entropy	长期接近 log L	head 未学习有效路由或被其他路径替代
V RMS / O_h RMS	head 间差异极端	value/out projection outlier、量化风险
W_O block norm	某些 head block 近 0	该 head 写回 residual 的作用弱
head gradient norm	长期接近 0	dead/redundant head 或优化问题
head-head similarity	大量 heads attention/outputs 高相似	冗余/未分化
max attention logit	持续增长	Q/K norm、precision 或 position scaling

21.2 推理 parity

- full-sequence forward vs token-by-token cached decode logits 必须在容差内一致。
- MHA baseline vs GQA converted model 先验证 shape/layout，再讨论质量。
- TP=1 vs TP=N 输出 parity。
- prefill 一次 vs chunked prefill 输出 parity。
- 不同 SDPA/Flash backend 做数值 parity 与性能分开验。

21.3 性能诊断顺序

- 1) confirm H_q/H_kv/d_head/config
- 2) confirm cache tensor shape + allocated bytes
- 3) confirm actual attention backend/kernel
- 4) profile HBM read + GEMM utilization
- 5) check TP/CP communication
- 6) sweep context and batch
- 7) only then compare architecture claims

22. Ablation Matrix : 怎样证明“这个 Head 配置更好”

Ablation	保持不变	改变	必须观察
Head count	params/token budget/optimizer	H_q, d_head 随之变化	loss、kernel 效率、head diversity
Head dim	d_model 尽量固定	显式 d_head + projection width	rank、稳定性、吞吐
KV heads	H_q/d_head/训练预算	H_kv	quality、KV bytes、TPOT
MHA→GQA	初始化/checkpoint/data	group mapping/uptrain budget	recovery curve
GQA + TP	模型权重/context	TP world size	KV 复制、通信、latency
RoPE × heads	数据/训练长度	rotary dims/theta/head_dim	long-context loss
Pruning	checkpoint/eval set	删除 head 集合	quality + real latency
MLA/GQA	总训练 tokens/宽深度	cache mechanism	quality/bytes/TP scaling

22.1 最低评测集合

- Validation loss / perplexity: 最敏感的基础信号。
- 短上下文 + 长上下文: 避免 cache 设计只在短任务看起来无损。
- 知识/推理/代码/多语言: head sharing 的损失可能非均匀。
- TTFT / TPOT / p95 / max batch: 系统结果。
- 实际 HBM bytes / KV allocated / communication: 解释为什么快或不快。

Know-how 先用 100M-1B proxy 搜索宽范围，再在接近目标规模做局部验证；但不要假设所有 head scaling 规律可完全从小模型外推。

23. PyTorch MVP : MHA / GQA / MQA

下面是概念性实现，强调 shape 与 KV sharing。生产环境应使用 fused SDPA/FlashAttention，并处理 cache、RoPE、mask、TP 等。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class GQAttention(nn.Module):
    def __init__(self, d_model, n_q, n_kv):
        super().__init__()
        assert d_model % n_q == 0
        assert n_q % n_kv == 0
        self.n_q, self.n_kv = n_q, n_kv
        self.dh = d_model // n_q
        self.q = nn.Linear(d_model, n_q * self.dh, bias=False)
        self.k = nn.Linear(d_model, n_kv * self.dh, bias=False)
        self.v = nn.Linear(d_model, n_kv * self.dh, bias=False)
        self.o = nn.Linear(n_q * self.dh, d_model, bias=False)

    def forward(self, x, is_causal=True):
        B, L, _ = x.shape
        q = self.q(x).view(B, L, self.n_q, self.dh).transpose(1, 2)
        k = self.k(x).view(B, L, self.n_kv, self.dh).transpose(1, 2)
        v = self.v(x).view(B, L, self.n_kv, self.dh).transpose(1, 2)

        # Reference-only logical expansion. Efficient kernels should avoid
        # physically materializing repeated K/V.
        repeat = self.n_q // self.n_kv
        k = k.repeat_interleave(repeat, dim=1)
        v = v.repeat_interleave(repeat, dim=1)

        y = F.scaled_dot_product_attention(q, k, v, is_causal=is_causal)
        y = y.transpose(1, 2).contiguous().view(B, L, -1)
        return self.o(y)

# MHA: n_q=n_kv
# MQA: n_kv=1
# GQA: 1<n_kv<n_q
```

23.1 必做 parity tests

```
# 1. shape
assert model(x).shape == x.shape

# 2. MHA special case
mha_like = GQAttention(d_model=512, n_q=8, n_kv=8)

# 3. MQA special case
mqa_like = GQAttention(d_model=512, n_q=8, n_kv=1)

# 4. full-forward vs cached-decode parity (add KV cache implementation)
# 5. TP=1 vs TP=N parity
# 6. math SDPA vs flash SDPA tolerance test
```

Gotcha 上面的 repeat_interleave 是教学写法。真正高效 GQA kernel 必须避免把 K/V 实际复制成 H_q 份，否则内存带宽收益会明显缩水。

24. 深度面试与系统设计题

1. 固定 d_{model} 时，为什么增加 head 数通常不改变 MHA 参数量？推导公式。
2. 为什么 head count 增加后 attention FLOPs 一阶项仍近似不变？
3. 多头相对单头的核心表达优势是什么？不要只回答“看不同信息”。
4. 为什么可以观察到 head specialization，同时又剪掉大多数 heads？
5. 为什么 attention heatmap 不能直接当特征重要性？
6. W_O 在 multi-head attention 中承担什么作用？删除它会怎样？
7. MQA 为什么只共享 K/V、不共享 Q？
8. 用 memory-bandwidth 模型解释 MQA/GQA 对 decode 的收益。
9. 推导 MHA/GQA/MQA 的 KV cache 字节数。
10. 32 Q heads、8 KV heads、TP=16 时，你会如何分片？有什么复制？
11. MHA checkpoint 如何初始化成 GQA？为什么还需要 uptraining？
12. 为什么 GQA 在 prefill 上未必有与 KV cache 相同倍数的加速？
13. 什么情况下更多 heads 反而可能变慢？从 kernel 角度回答。
14. 为什么改变 head_dim 可能连带破坏 RoPE 配置？
15. Head pruning 如何才能真正得到 wall-clock speedup？
16. 解释 MHA→GQA→MQA→MLA 的“缓存状态压缩”连续谱。
17. MLA 为什么需要位置相关分支与 latent content 分离？
18. 如果 GQA 质量下降只发生在代码任务，你如何诊断？
19. 设计一个 head-count/ H_{kv} 的 proxy-model ablation。
20. 为什么 TP/CP 拓扑属于 attention architecture 的一部分？
21. 你如何判断某个 head 是“冗余”还是被其他 head 补偿？
22. KV cache 量化为什么应该按 head/层分析 outlier？
23. 如果配置声明 GQA，但实际显存没有降低，你会按什么顺序排查？
24. 如何验证 fused QKV 的 split 顺序没有错？
25. 2026 年面对 GQA、MLA、新型低秩 KV，你会怎样做生产选型？

25. 参考文献与调研来源

- [1] Vaswani et al. Attention Is All You Need. NeurIPS 2017. arXiv:1706.03762.
- [2] Michel, Levy, Neubig. Are Sixteen Heads Really Better than One? NeurIPS 2019. arXiv:1905.10650.
- [3] Voita et al. Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned. ACL 2019. arXiv:1905.09418.
- [4] Clark et al. What Does BERT Look At? An Analysis of BERT's Attention. 2019. arXiv:1906.04341.
- [5] Shazeer. Fast Transformer Decoding: One Write-Head is All You Need. 2019. arXiv:1911.02150.
- [6] Ainslie et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. EMNLP 2023. arXiv:2305.13245.
- [7] Bhojanapalli et al. Low-Rank Bottleneck in Multi-head Attention Models. ICML 2020. arXiv:2002.07028.
- [8] Shazeer et al. Talking-Heads Attention. 2020. arXiv:2003.02436.
- [9] Touvron et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. 2023. arXiv:2307.09288.
- [10] Jiang et al. Mistral 7B. 2023. arXiv:2310.06825.
- [11] Yang et al. Qwen2 Technical Report. 2024. arXiv:2407.10671.
- [12] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. arXiv:2405.04434.
- [13] Xiao et al. Improving Transformers with Dynamically Composable Multi-Head Attention. 2024. arXiv:2405.08553.
- [14] DHA: Learning Decoupled-Head Attention from Transformer Checkpoints. 2024. arXiv:2406.06567.
- [15] MoH: Multi-Head Attention as Mixture-of-Heads. 2024. arXiv:2410.11842.
- [16] Cost-Optimal Grouped-Query Attention for Long-Context LLMs. 2025. arXiv:2503.09579.
- [17] TransMLA: Multi-Head Latent Attention Is All You Need. 2025. arXiv:2502.07864.
- [18] The Effect of Attention Head Count on Transformer Scaling / Expressivity. 2025-2026 preprint, arXiv:2510.06662.
- [19] Low-Rank Key Value Attention (2026, arXiv:2601.11471) and Multi-Head Low-Rank Attention (2026, arXiv:2603.02188) - emerging work.
- [20] Jain, Wallace. Attention is not Explanation. NAACL 2019. arXiv:1902.10186.
- [21] Grattafiori et al. The Llama 3 Herd of Models. 2024. arXiv:2407.21783.

证据分层 [1]-[12] 主要用于成熟机制和已广泛复现的工程结论；[13]-[19] 属于较新的 attention-head/低秩方向，报告只作为研究前沿介绍，结论不视为统一生产最佳实践。

最终总结

理解 Multi-Head Attention 的最好方式不是记住 “H 个 attention 并行”，而是把它看成一个预算分配问题：在固定 residual width 下，把匹配几何拆成多少个子空间；历史 token 需要保存多少个 K/V 状态；这些状态怎样被 GPU 分片和搬运；以及不同 head 的输出怎样通过 W_O 写回共享 residual stream。MHA→MQA→GQA→MLA 的演化，就是在不断重新回答这个问题。